CHAUDHARY CHARAN SINGH UNIVERSITY (CCSU) • BCA V SEMESTER

BCA-501: Introduction to DBMS

Session: Comprehensive PYQ Bank Examination | **Paper Code:** 18021 | **Complete 100% Solved Paper (All 13 Questions)** | **Time:** 3 Hours • **Max Marks:** 75

SECTION – A (VERY SHORT ANSWER QUESTIONS • 3 MARKS EACH • ALL 5 COMPULSORY)

Q1. Define Database Management System (DBMS) and its fundamental purpose. [3 Marks]

Answer:

A **Database Management System (DBMS)** is a specialized software package designed to define, construct, manipulate, and share structured databases among various users and applications.

> **Four Fundamental Purposes of DBMS:**
> 1. **Controlled Redundancy:** Integrates multiple logical files to eliminate duplicate data storage.
> 2. **Data Consistency & Integrity:** Enforces business constraints and ACID properties across operations.
> 3. **Concurrent Multi-User Access:** Allows hundreds of simultaneous users to access data without conflicts.
> 4. **Data Security & Recovery:** Provides role-based access control, transaction logs, and disaster recovery.

Q2. Distinguish between Simple and Composite Attributes with examples. [3 Marks]

Answer:

Attribute Type	Definition & Characteristic	Real-World Examples
Simple Attribute (Atomic)	Cannot be further divided into smaller independent sub-components. Holds a single atomic value.	`Roll_No`, `Age`, `Gender`, `Salary`
Composite Attribute	Can be divided into smaller sub-attributes, each having independent semantic meaning.	`Name` → (First, Middle, Last) `Address` → (Street, City, State, PIN)

Q3. Explain Referential Integrity Constraint. [3 Marks]

Answer:

Referential Integrity is a relational constraint between two relations (tables) maintained through a **Foreign Key**. It dictates that:

> A value in the Foreign Key column of the referencing table **MUST** either match an existing Primary Key value in the referenced table, or otherwise be `NULL`.

- **Prevents Orphan Records:** Prevents child records from pointing to non-existent parent rows.
- **Cascading Actions:**
  - `ON DELETE CASCADE`: Deleting a parent row automatically deletes all matching child rows.
  - `ON DELETE SET NULL`: Deleting a parent row resets foreign key fields in child rows to `NULL`.

Q4. Differentiate between DDL and DML commands with syntax. [3 Marks]

Answer:

Feature	DDL (Data Definition Language)	DML (Data Manipulation Language)
Primary Objective	Defines, alters, and drops database schema structures and tables.	Manages, inserts, updates, and retrieves data tuples within existing tables.
Transaction Behavior	**Auto-committed** immediately; changes cannot be rolled back via TCL.	**Not auto-committed** by default; can be committed or rolled back using `ROLLBACK`.
Key SQL Commands & Syntax	`CREATE TABLE Student(Id INT);` `ALTER TABLE Student ADD Age INT;` `DROP TABLE Student;` `TRUNCATE TABLE Student;`	`INSERT INTO Student VALUES(1, 20);` `UPDATE Student SET Age=21 WHERE Id=1;` `DELETE FROM Student WHERE Id=1;` `SELECT * FROM Student;`

Q5. Explain Functional Dependency (FD) and Multivalued Dependency (MVD). [3 Marks]

Answer:

1. Functional Dependency (X → Y):

A constraint between two sets of attributes X and Y in relation R. It states that for any two tuples t_1 and t_2, if $t_1[X] = t_2[X]$, then $t_1[Y] = t_2[Y]$ must also hold. Attribute set X uniquely determines attribute set Y.

Example: $Roll_No \rightarrow Student_Name, Branch$

2. Multivalued Dependency (X ↠ Y):

Occurs when presence of one or more rows in a table implies the presence of other rows. For a tuple (x, y, z), attribute set Y depends only on X and is completely independent of attribute set Z. MVD leads to **4NF decomposition**.

Example: In table (Course, Teacher, Book), if Course determines multiple Teachers and multiple Books independently: $Course \twoheadrightarrow Teacher$ *and* $Course \twoheadrightarrow Book$.

SECTION – B (SHORT ANSWER QUESTIONS • 7.5 MARKS EACH • ATTEMPT ANY TWO)

Q6. Discuss Anomalies associated with Relational Database (Insertion, Deletion, Update). [7.5 Marks]

Answer:

When relational schemas are unnormalized, redundant information causes three major operational anomalies:

Consider Unnormalized Relation: `Emp_Dept(EmpId, EmpName, DeptId, DeptName, DeptHOD)`

1. **Insertion Anomaly:**
   Occurs when certain facts cannot be recorded without prematurely adding unrelated attributes. For example, a new department (e.g., DeptId = 'D99', DeptName = 'AI Research') cannot be inserted until at least one employee is hired, because `EmpId` is the primary key and cannot accept `NULL`.
2. **Deletion Anomaly:**
   Occurs when deleting one piece of data inadvertently destroys critical unrelated information. For example, if employee 'E101' is the sole member of department 'D05', deleting employee 'E101' simultaneously deletes all records of department 'D05' (its name, budget, HOD) from the database.
3. **Update (Modification) Anomaly:**
   Occurs when data redundancy causes duplicate copies across multiple tuples. If department 'D01' has 500 employees, changing the department name requires updating all 500 rows. Missing even one row results in conflicting, inconsistent database states.

Solution: Normalization decomposes such relations into separate tables (e.g., `Employee` and `Department`) linked by foreign keys.

Q7. Explain Super Key, Candidate Key, and Primary Key with suitable examples. [7.5 Marks]

Answer:

Consider relation: `Student(RollNo, RegNo, Email, Name, Age, Phone)`

> **Hierarchical Relationship:** Super Key &supd; Candidate Key &supd; Primary Key

Key Category	Formal Definition & Properties	Examples from Student

Super Key	Any set of one or more attributes that uniquely identifies each tuple in a relation. May contain redundant attributes.	`{RollNo}`
		`{RegNo}`
		`{Email}`
		`{RollNo, Name}`
		`{RegNo, Age, Phone}`

Candidate Key	A **minimal** super key. Contains no extraneous attributes; removing any single attribute destroys unique identification.	`{RollNo}`
		`{RegNo}`
		`{Email}`

Primary Key	The single candidate key chosen by the database designer as the principal unique identifier. **Cannot be NULL** and must be unique.	`{RollNo}` (Chosen as primary key)

Q8. What is Normalization? Explain Second Normal Form (2NF) and Third Normal Form (3NF). [7.5 Marks]

Answer:

Normalization is a systematic technique for structuring relational tables to eliminate data redundancy, prevent insertion/deletion/update anomalies, and preserve data integrity.

> **1. Second Normal Form (2NF):**
>
> - **Condition 1:** Relation must already satisfy First Normal Form (1NF) — all attributes atomic.
> - **Condition 2: No Partial Dependency:** Every non-prime attribute must depend on the *entire* candidate key, not on any proper subset of a composite candidate key.
>
> *Example: In* `StudentCourse(StudentID, CourseID, Grade, CourseName)` *with candidate key* `{StudentID, CourseID}` *:*
>
> `CourseID → CourseName` is a partial dependency. Resolved by decomposing into: `Enrollment(StudentID, CourseID, Grade)` and `Course(CourseID, CourseName)`.

> **2. Third Normal Form (3NF):**
>
> - **Condition 1:** Relation must already be in 2NF.
> - **Condition 2: No Transitive Dependency:** Non-prime attributes must not depend on other non-prime attributes. For every non-trivial FD `X → Y`:
> - Either **X is a Super Key**, OR
> - **Y is a Prime Attribute** (part of any candidate key).
>
> *Example: In* `Employee(EmpId, EmpName, DeptId, DeptName)` *where* `EmpId → DeptId` *and* `DeptId → DeptName` *:*
> Decompose into `Emp(EmpId, EmpName, DeptId)` and `Dept(DeptId, DeptName)`.

SECTION – C (LONG ANSWER QUESTIONS • 15 MARKS EACH • ATTEMPT ANY THREE)

Q9. Define Boyce-Codd Normal Form (BCNF). Compare 3NF and BCNF with a suitable example. [15 Marks]

Answer:

Boyce-Codd Normal Form (BCNF) is a stricter, stronger version of 3NF. A relation R is in BCNF if and only if:

> For every non-trivial Functional Dependency `X → Y` in R, **X MUST be a Super Key**.

Key Difference between 3NF and BCNF:

Property	Third Normal Form (3NF)	Boyce-Codd Normal Form (BCNF)
Condition for X → Y	Valid if X is Super Key **OR** Y is a prime attribute.	Valid **ONLY** if X is a Super Key (no relaxation for prime Y).
Dependency Preservation	Always preserves all functional dependencies.	May occasionally sacrifice dependency preservation.
Redundancy Level	May retain slight redundancy for overlapping keys.	Strictly zero redundancy caused by functional dependencies.

Classic Example Illustrating 3NF vs BCNF:

Consider relation `StudentAdvisor(StudentID, Subject, Advisor)` with business rules:

- A student can take multiple subjects and have an advisor for each: `{StudentID, Subject} → Advisor`
- Each advisor teaches only one subject: `Advisor → Subject`
- Candidate Keys: `{StudentID, Subject}` and `{StudentID, Advisor}`.
- Prime Attributes: `StudentID`, `Subject`, `Advisor`.

> **Check for 3NF:** For FD `Advisor → Subject`, Advisor is not a super key, but **Subject is a prime attribute**. Therefore, the table is in **3NF** ✓.
>
> **Check for BCNF:** In `Advisor → Subject`, Advisor is **NOT** a super key. Hence, this table **violates BCNF** ✗.
>
> **BCNF Decomposition:** Split into two relations:
>
> 1. `AdvisorSubject(Advisor, Subject)` — Key: `Advisor` ✓
> 2. `StudentAdvisor(StudentID, Advisor)` — Key: `{StudentID, Advisor}` ✓

Q10. Explain Relational Algebra Fundamental Operations with examples. [15 Marks]

Answer:

Relational Algebra is a procedural query language consisting of operations that take one or two relations as input and produce a new relation as output.

Operation & Symbol	Description & Formal Rule	Concrete Example Expression
Selection (σ)	Selects tuples that satisfy a given predicate condition. Acts as a horizontal filter.	$\sigma_{Marks > 75}(Student)$ Filters students scoring above 75.
Projection (π)	Selects specified attribute columns and automatically discards duplicate tuples. Vertical filter.	$\pi_{RollNo, Name}(Student)$ Extracts RollNo and Name columns.
Cartesian Product (×)	Combines each tuple of relation R with every tuple of relation S. Output size: $\|R\| \times \|S\|$.	$Student \times Course$
Union (∪)	Combines unique tuples from two union-compatible relations (identical arity and compatible domains).	$\pi_{Name}(Faculty) \cup \pi_{Name}(Student)$
Set Difference (−)	Returns tuples present in relation R but not in relation S (must be union-compatible).	$\pi_{RollNo}(AllStudents) - \pi_{RollNo}(Placed)$
Rename (ρ)	Renames a relation or its attributes to prevent ambiguities in self-joins.	$\rho_{S1}(Student)$
Natural Join (⋈)	Equi-join combining Cartesian product, equality selection on common attributes, and duplicate column removal.	$Student \bowtie Department$

Q11. Explain SQL Characteristics and Five Aggregate Functions with SQL queries. [15 Marks]

Answer:

SQL Characteristics: SQL is a declarative, non-procedural language allowing users to specify *what* data is needed rather than *how* to retrieve it. It integrates DDL, DML, DCL, and TCL commands with full ACID compliance.

Five Core Aggregate Functions (operating on Employee table):

Function	Purpose	SQL Query Example & Description
`COUNT()`	Counts number of non-null values or total rows matching criteria.	`SELECT COUNT(*) FROM Employee WHERE Dept = 'IT';` Returns total number of employees in IT department.
`SUM()`	Calculates total arithmetic sum of numeric values.	`SELECT SUM(Salary) FROM Employee WHERE Dept = 'Sales';` Calculates total monthly payroll for Sales department.
`AVG()`	Calculates average mean value of numeric attribute.	`SELECT AVG(Salary) FROM Employee;` Returns average company-wide employee salary.
`MIN()`	Returns smallest minimum value in the attribute set.	`SELECT MIN(Salary) FROM Employee;` Finds the lowest compensation in the firm.
`MAX()`	Returns highest maximum value in the attribute set.	`SELECT Dept, MAX(Salary) FROM Employee GROUP BY Dept;` Returns highest salary earned in each department.

Q12. Explain Concurrency Control and Two-Phase Locking (2PL) Protocol with its pitfalls. [15 Marks]

Answer:

Concurrency Control coordinates simultaneous transaction execution to prevent conflicts (Lost Update, Uncommitted Read, Inconsistent Analysis) and ensure Serializable schedules.

Two-Phase Locking (2PL) Protocol:

1. **Growing (Expansion) Phase:** Transaction may acquire locks (Shared `S-Lock` or Exclusive `X-Lock`) but **cannot release any lock**.
2. **Lock Point:** The moment the transaction has acquired all necessary locks.
3. **Shrinking (Contraction) Phase:** Transaction may release locks but **cannot acquire any new locks**.

Theorem: Any schedule following the 2PL protocol is guaranteed to be **Conflict Serializable**.

Pitfalls of Basic 2PL & Solutions:

- **Cascading Aborts / Rollbacks:** If T1 releases a lock early in its shrinking phase and then aborts, any transaction T2 that read T1's dirty data must also abort.
  Solution → Strict 2PL: All Exclusive (X) locks must be held until the transaction commits or aborts.
- **Deadlocks:** 2PL does not prevent cyclic wait conditions where T1 holds Lock A waiting for Lock B, while T2 holds Lock B waiting for Lock A.
  Deadlock Handling Mechanisms:
  - *Deadlock Detection:* Maintains a dynamic **Wait-For Graph (WFG)**; cycles indicate deadlock and trigger victim abort.
  - *Deadlock Prevention:* Uses transaction timestamps: **Wait-Die** (non-preemptive) or **Wound-Wait** (preemptive).

Q13. Write detailed notes on: (a) Three-Schema Architecture, (b) Data Independence, and (c) Role of DBA. [15 Marks]

Answer:

(a) Three-Schema (ANSI/SPARC) Architecture:

1. **External Level (View Level):** Multiple tailored views for end-users and applications; conceals all irrelevant details and restricts unauthorized access.
2. **Conceptual Level (Logical Level):** Community-wide global schema describing what data is stored, entities, relationships, attributes, and constraints (tables, primary/foreign keys).
3. **Internal Level (Physical Level):** Physical data storage on disk, block allocation, record layout, B-Tree indexes, hashing, and encryption algorithms.

(b) Data Independence:

Independence Level	Definition & Mechanism	Real-World Scenario
Physical Data Independence	Ability to alter physical schema (disk storage, indexes, RAID, hashing) without modifying logical/conceptual schema.	Adding a B-Tree index on `Salary` to boost search speed without altering table structure.
Logical Data Independence	Ability to alter conceptual schema (add tables, add columns) without modifying existing external schemas or application programs.	Adding a `PassportNo` column to Student table without breaking existing grade-card report queries.

(c) Role and Responsibilities of Database Administrator (DBA):

- **Schema Definition & Storage Structure:** Creates logical tables and allocates physical disk storage paths.
- **Security & Authorization:** Grants and revokes permissions (`GRANT` , `REVOKE`) to prevent data leaks.
- **Performance Tuning:** Monitors slow queries, builds indexes, and reorganizes partitions.
- **Backup & Disaster Recovery:** Schedules incremental/full backups and handles crash recovery using transaction logs.